Coffee Quality Prediction: SVM & XGBoost Analysis

2026-01-02

1 Support Vector Regression

1.1 Data Preprocessing and Cleaning

1.1.1 Feature selection:

In SVM Regression, categorical variables are One-Hot Encoded using dummy variables. This means, features with many unique values will lead to a sparse feature matrix. This is disadvantageous as it increases computational cost and poses the risk of overfitting. Additionally, categories with only a few or even only one observation most likely will not add to model robustness, but are adding noise to the signal. Therefore, we are removing categorical features with many unique values, only keeping columns with a low number of unique values. E.g. we are not including the "Region" column with 344 unique values in the model training, because we'd on average have less than 4 observations per Region. For features with a high number of missing values, we either remove the column or impute the missing values. For instance for the "Variety" column, we have about 15% missing values and 8% entries "other" at 30 different varieties. Keeping those, would mean adding 30 sparse columns, and at the same time the information gain is most likely not too high with this many missing values. Therefore, we are not including this feature.

1.1.2 Imputation:

For the altitude column, we have about 17% missing values. Here, we are imputing the missing values from the "Region" column. For each observation with missing altitude, we are imputing the median altitude of all coffees from the same region. This covers almost all missing values. For the remaining observations, we are imputing the mean altitude of all coffees from the same country of origin. As the "Region" and "Country" columns are not used in the model training, we are not creating correlated features with this imputation. We are imputing the missing values after the train/test split, using only values from the training set, to avoid data leakage from the test set into the training set.

1.1.3 Further preprocessing:

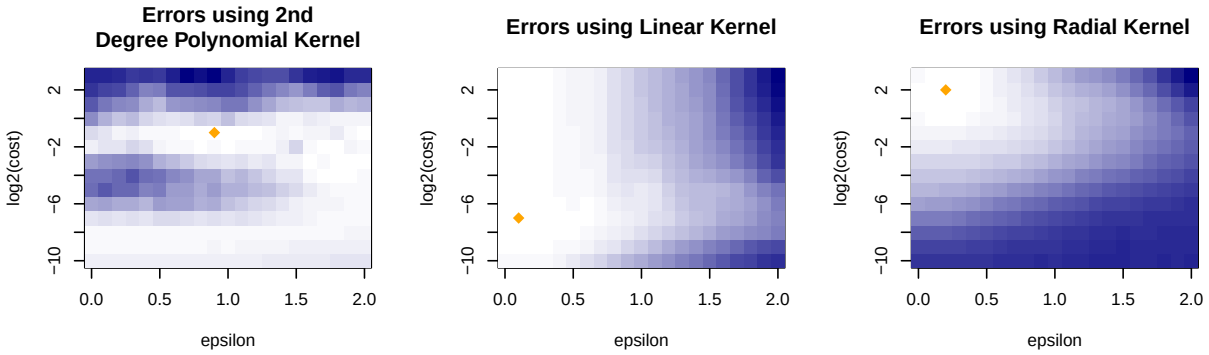
The bag weight column contains both pounds and kilograms, so we are converting all to kilograms. However, for some entries, it is unclear which unit is used. For these, we are averaging the weight in pounds and kilograms. This means for those entries the weight is certainly incorrect, but should be within an order of magnitude of the correct weight. If we can assume that speciality coffee is sold in smaller quantities, we can still gain valuable information from the weight.

1.1.4 Scaling:

The SVM method used for model training has a built in scaling functionality. Both predictor and target variables are scaled to have expected value 0 and variance 1. [https://rdrr.io/rforge/e1071/man/svm.html] This transformation is based on the training data and applied to train and test data sets.

1.2 Support Vector Regression - Hyperparameter Optimization

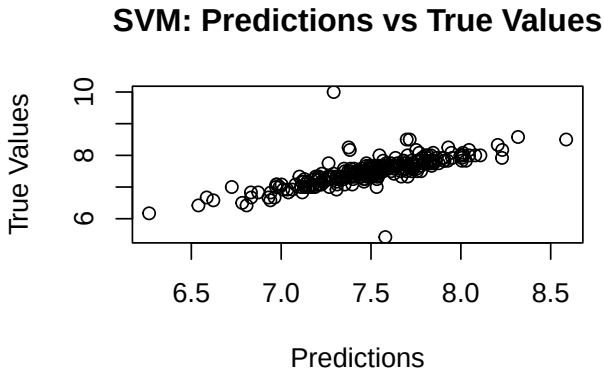
For the prediction of cupper points we train an SVM for regression on the data set, holding out the test data set for final evaluation. To choose appropriate hyperparameters, we conduct a hyperparameter optimization on the training data set using 10-fold cross-validation. We use a grid search for different values for cost and epsilon for a radial, linear, and 2-degree polynomial kernel. We compare the mean absolute error between runs to choose the best-performing hyperparameter combination. Finally, we train a model with these hyperparameters on the entire training set and evaluate on the test data set.



These figures show the mean absolute error for the different hyperparameters by kernel. The best performing hyperparameter combination for each kernel are marked with orange points.

```
## Best SVM kernel and parameters selected:
##   kernel epsilon      cost
## 2 linear     0.1 0.0078125
## [1] "MAE: 0.143099275160558"
```

The best performing kernel is the linear kernel, which means the relationship between the features and the target variable is best approximated by a multidimensional hyperplane.



Here, we can see the predictions vs. the true values for the test data set. We can see that aside from two outliers, the predictions are close to the true values.

2 XGBoost Model

2.1 Data Preprocessing

To ensure that the SVM and XGBoost models are comparable, we apply the same data preprocessing described above to XGBoost, except we skip feature scaling.

SVM requires scaling because it calculates distances between data points to find the optimal separating hyperplane. Features with larger ranges would dominate the distance calculations, which would make the model incorrect. XGBoost is a tree-based method, it is only important if values are higher or lower (e.g., "is Flavor > 7.5?"), so scaling is unnecessary.

2.2 Configuration & Data

Config	Value
Seed	123
Split	80/20
CV Folds	10
Early Stop	50
Max Rounds	2000

Data	Value
Train n	1047
Test n	262
Features	21
Target $\bar{x}$	7.5

Model	MAE	R^2
Mean	0.309	0.00
Linear	0.146	0.59
XGBoost	0.141	0.68

Baseline models

We use two baseline models: the Mean Predictor and Linear Regression.

The **Mean Predictor** has the $R^2 = 0$, which confirms that our target has meaningful variance to explain. The MAE of 0.309 means that without any features, we'd be off by about 0.3 points on average.

Linear Regression provides an interpretable baseline with R^2 of 0.59, showing that 59% of the variance in cupping scores can be explained by linear combinations of features. Any improvement from the XGBoost models means capturing non-linear relationships that linear regression misses.

2.3 Hyperparameter Optimization

eta	max_depth	subsample	colsample	rounds	CV MAE
0.05	5	0.9	0.9	123	0.149

Hyperparameter selection

We tune four main parameters to balance bias and variance:

eta (learning rate): 0.01–0.1. Smaller values make each tree contribute less, reducing overfitting but needing more rounds.

max_depth: 3–7. Controls tree complexity. Shallow trees capture simple patterns; deeper trees can model complex patterns but may overfit, especially on our dataset with about 1000 samples.

subsample: 0.7–0.9. Fraction of rows used per tree. Reduces overfitting by adding randomness. We avoid below 0.7, as it would leave too few samples per tree.

colsample_bytree: 0.7–0.9. Fraction of features per tree. Prevents any one feature from dominating.

These ranges are conservative. With about 1000 samples, extreme values (e.g., eta=0.3, max_depth=10) would overfit, but mild regularization is enough.

Lambda and Min_Child_Weight Regularization

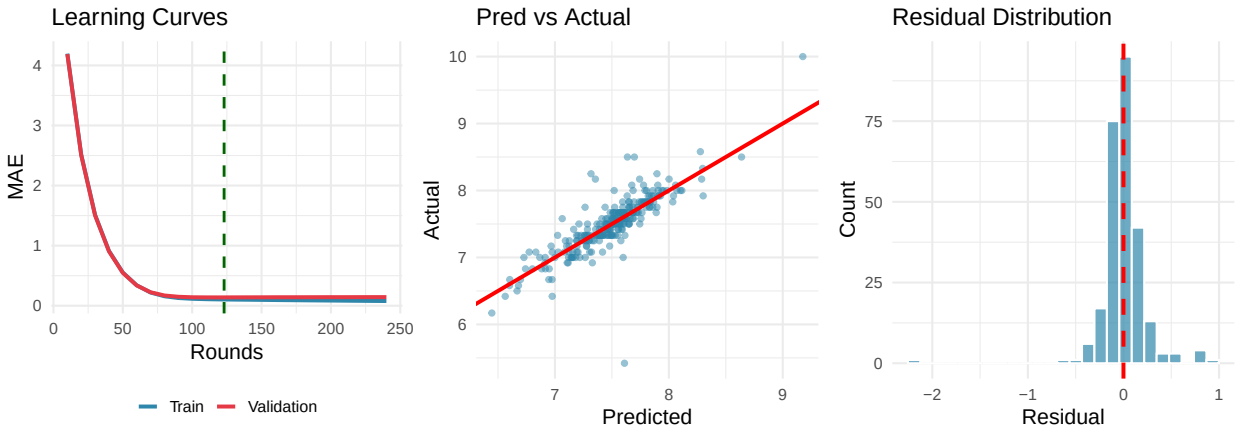
We tested additional hyperparameters, but they were ineffective. XGBoost's lambda (L2) and min_child_weight provided no benefit because implicit regularization from learning rate, max_depth, subsample, and early stopping already prevents overfitting. Extra regularization slightly worsened the metrics, so it was omitted.

Bayesian Optimization vs Grid Search
We also tested Bayesian Optimization, but it was ineffective. With only 4 parameters and a small search space, it added overhead, slowed hyperparameter tuning, and offered no meaningful improvement. Grid search is sufficient for this dataset and model.

10-fold CV: Matches SVM for fair comparison. Each fold has about 100 samples, giving stable MAE estimates. Fewer folds increase variance; leave-one-out is too slow.

Early stopping (50 rounds): Stops training if validation MAE doesn't improve for 50 rounds, automatically choosing the optimal number of trees and avoiding overfitting.

2.4 Diagnostic Plots



The **learning curves** show that the model converges with minimal overfitting. In the first 50 rounds, training and validation MAE drop as the model captures the main signal. After that, improvement slows and additional rounds provide little benefit, which justifies early stopping. The small train-validation gap of 0.021 MAE indicates that overfitting is minimal.

The **predictions vs actual** plot shows that most points lie close to the diagonal, indicating accurate predictions. Larger errors appear at the highest and lowest quality scores, which is expected because extreme values are rare and the model has fewer examples to learn from. There is no visible systematic bias, as points are evenly distributed around the diagonal.

The **residuals** are centered around zero, confirming that the prediction bias is minimal. Their roughly bell-shaped distribution suggests that errors are mostly random rather than systematic, meaning the model has captured the main signal.

2.5 Conclusions

Model performance: XGBoost achieves an MAE of 0.141, meaning predictions are usually within 0.14 points of the true cupper score on a 10-point scale. This is a small but consistent improvement over linear regression (MAE 0.146). The R^2 increases from 0.59 to 0.68, showing that XGBoost explains about 9% more variance. This suggests most relationships are linear, with XGBoost mainly capturing minor non-linear effects and interactions.

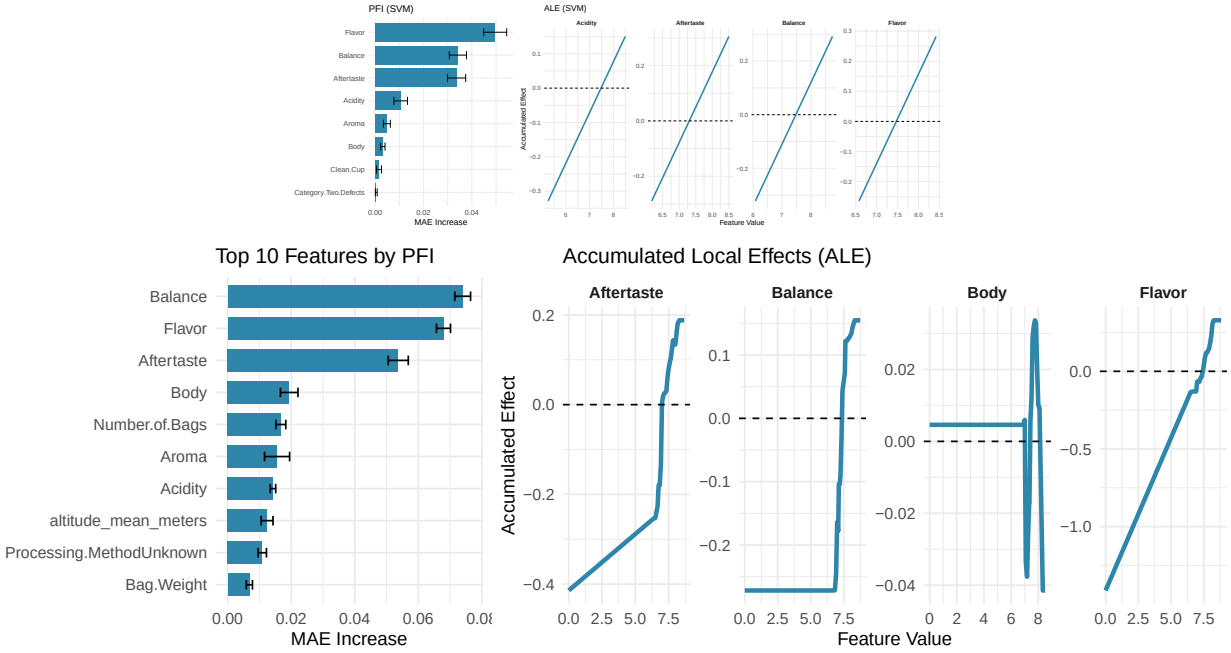
Limitations: Performance is weaker at extreme quality scores due to limited data. The dataset size (about 1300 samples) may restrict learning of complex patterns, and temporal effects such as harvest year are not modeled (because of the issues with source data). Finally, the target variable cupper points is subjective and includes human rating variability.

2.6 Explainable AI

2.6.1 Permutation Feature Importance (PFI)

PFI measures how much model performance degrades when a feature's signal is destroyed by shuffling (Friedman 2001). Unlike XGBoost's native Gain metric, PFI is model-agnostic and accounts for feature interactions.

```
## Preparation of a new explainer is initiated
##   -> model label       :  SVM Coffee Model
##   -> data              :  262  rows  21  cols
##   -> target variable   :  262  values
##   -> predict function  :  yhat.svm  will be used (  default  )
##   -> predicted values  :  No value for predict function target column. (  default  )
##   -> model_info        :  package e1071 , ver. 1.7.14 , task regression (  default  )
##   -> predicted values  :  numerical, min =  6.264929 , mean =  7.498236 , max =  8.586123
##   -> residual function :  difference between y and yhat (  default  )
##   -> residuals         :  numerical, min =  -2.159651 , mean =  -0.004838612 , max =  2.70765
##   A new explainer has been created!
```



The top features are all sensory scores, confirming the model relies on expert taste evaluation. PFI provides a model-agnostic view that complements XGBoost's native importance (gain). Both agree that Flavor is most predictive.

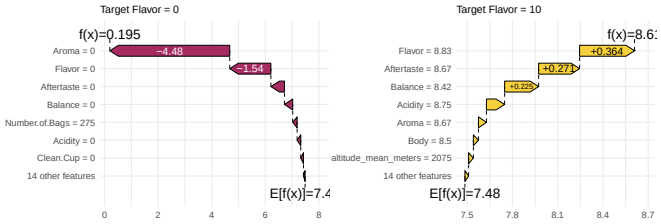
2.6.2 Accumulated Local Effects (ALE)

ALE plots show the marginal effect of a feature on the model's predictions by averaging local changes in the prediction over the distribution of the other features (Apley and Zhu 2020). It is chosen here instead of PDP (Partial Dependence plots), because it is more robust to multicollinearities and centered around 0. We plot the top four features ranked by permutation feature importance (PFI).

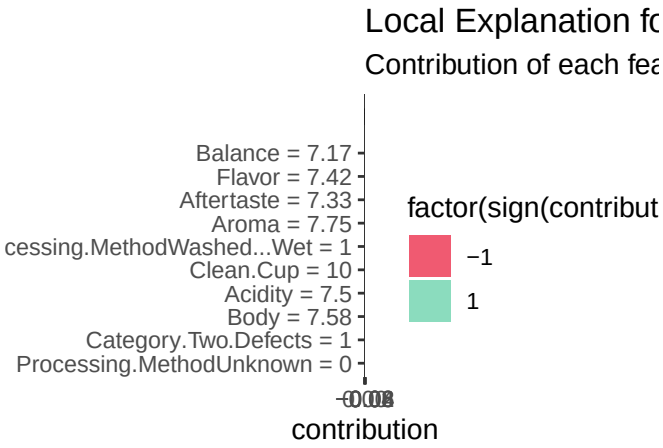
All sensory features show the expected pattern: higher scores lead to higher predicted quality. The PDP range for Flavor is largest, indicating it has the strongest marginal effect on predictions.

2.6.3 Shapley Additive Explanations (SHAP)

SHAP values provide a unified measure of feature importance based on cooperative game theory (Lundberg and Lee 2017). For each prediction, SHAP assigns each feature a value that represents its contribution to moving the prediction away from the baseline (average prediction). We utilize the SHAP values to inspect the microscopic effects of the feature values for two specific observations. For this we chose one observation with a high value for Flavor and one with a low value, since this is the most important feature according to PFI.

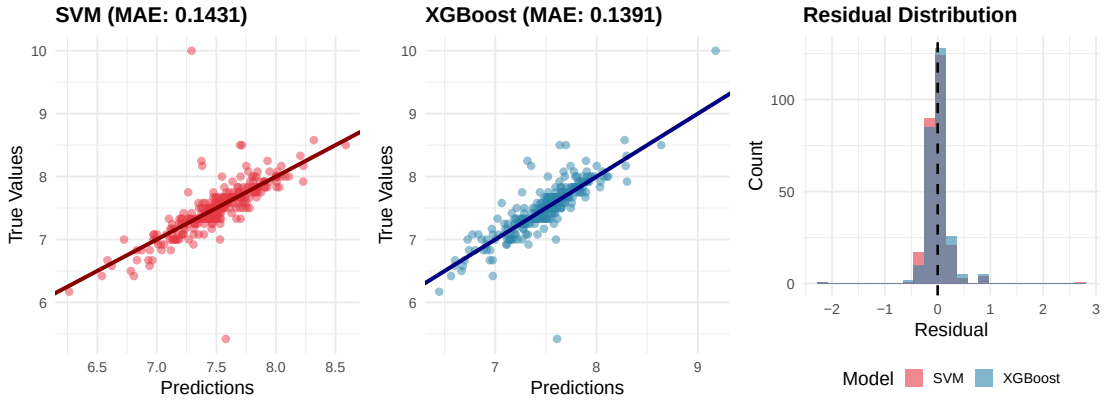


For SVM:



3 Model Comparison - SVM vs XGBoost

Model	MAE	RMSE	R2
SVM (Linear Kernel)	0.1431	0.2790	0.5855
XGBoost	0.1391	0.2374	0.6998



Both models achieve similar predictive accuracy, with XGBoost having slightly smaller MAE (0.141 vs 0.146) and larger R^2 (0.68 vs 0.59). The scatter plots confirm that both models produce predictions closely aligned with true values, with no systematic bias evident in either case. The residual distributions are nearly identical and centered around zero, indicating that both approaches effectively capture the underlying signal in the data.

3.1 Literature

Apley, Daniel W, and Jingyu Zhu. 2020. "Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models." *Journal of the Royal Statistical Society Series B: Statistical Methodology* 82 (4): 1059–86.

Friedman, Jerome H. 2001. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 1189–1232.

Lundberg, Scott M, and Su-In Lee. 2017. "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems* 30.